# Effective Exception Handling in Python for Data Science and Machine Learning Applications

**Course Name: Programming With Python DLMDSPWP01**

Type of Thesis: Research & Practical
Date: 13 March 2026
Author Name: Zaiba Khan
Author Email: zaiba.khan@iu-study.org
Matriculation Number: 4243508
Tutor Name: Cosmina Croitoru & Tai Le Quy

# Table of Contents


***Abstract*** ..... ***3***

***1. Introduction*** ..... ***4***

**1.1 Rationale and Relevance of the Topic** ..... 4

**1.2 Aim and Research Objectives** ..... 4

1**.3 Scope and Structure of the Assignment** ..... 5

***2. Fundamentals of Exception Handling in Python*** ..... ***5***

2.1 Concept of Exceptions in Programming ..... 5

2.2 Python's Exception Hierarchy ..... 5

2.3 Syntax and Core Constructs ..... 6

2.4 Custom Exceptions ..... 7

***3. Errors Sources in Data Science and Machine Learning Workflows*** ..... ***7***

3.1 Data Collection and Ingestion Errors ..... 7

3.2 Data Preprocessing and Feature Engineering Errors ..... 8

3.3 Model Training and Evaluation Errors ..... 8

3.4 Deployment and Production Errors ..... 8

3.5 Illustrative Python Code Examples for Common Error Scenarios ..... 8

***4. Best Practices for Effective Exception Handling in Python*** ..... ***10***

4.1 Catching Specific Exceptions ..... 10

4.2 Avoiding Silent Failures ..... 10

4.3 Using Exceptions for Exceptional Conditions ..... 11

4.4 Integrating Validation and Exception Handling ..... 11

***5. Beneficial Exception Handling in Machine Learning Libraries*** ..... ***11***

5.1 NumPy and SciPy ..... 11

5.2 pandas ..... 11

5.3 scikit-learn ..... 11

5.4 Deep Learning Frameworks ..... 12

***6. Logging, Monitoring, and Production Considerations*** ..... ***12***

6.1 Advanced Exception Handling with Logging (along with Production-Oriented Example) ..... 12

6.2 Exception Handling in Deep Learning Training Loops ..... 13

6.3 Supporting Reproducibility Through Controlled Failure Handling ..... 14

***7. Ethical, Reproducibility, and Scientific Implications*** ..... ***15***

***8. Conclusion*** ..... ***16***

***9 References*** ..... ***17***

***10 APPENDIX*** ..... ***18***

## Abstract

To create reliable, maintainable, and trustworthy data science and machine learning (ML) systems, effective exception management is an essential but frequently overlooked component. Understanding how to methodically handle errors, anomalies, and unexpected runtime conditions is crucial for guaranteeing data integrity, model dependability, reproducibility, and ethical deployment because **Python** has emerged as the most popular programming language in data-driven fields. Effective Python exception handling techniques are examined in this paper, with an emphasis on data science and machine learning applications. It covers recommended practices, theoretical underpinnings, Python-specific techniques, and practical applications throughout the machine learning lifecycle, from data intake to model deployment. To show how structured exception handling supports software quality, experimental validity, and operational resilience, the study synthesizes scholarly literature, official documentation, and industry practices.

**Keywords:** Python, exception handling, data science, machine learning, robustness, software reliability

# 1. Introduction

## 1.1 Rationale and Relevance of the Topic

Data science and machine learning systems dominates the high-stakes domains such as finance, healthcare, public policy, and transportation. Errors may occur discreetly during data preprocessing, model training, or deployment, resulting in inaccurate forecasts, skewed results, or system breakdowns. Recent research shows that data and software engineering problems, such as unregulated exceptions and poor error management, account for a significant portion of ML system failures rather than algorithmic defects (Sculley et al., n.d.).

Python's extensive ecosystem, robust community, and easy accessibility have rendered it the de facto standard language for data science and machine learning (Rossum & Drake, 2002; Jake VanderPlas, 2016). However, Python’s adaptability may also promote ad hoc coding techniques, such as ignoring exceptions, using excessively general error handling, or silently suppressing failures. The fundamental principles of scientific computing—reproducibility and reliability—are undermined by such approaches. (Wilson et al., 2014).

In consideration of this, efficient exception handling is a methodological prerequisite for ethical data science rather than just a programming issue. As a result, this subject is extremely significant to both professional practice and academic studies.

## 1.2 Aim and Research Objectives

The goal of this assignment is to critically evaluate effective exception handling in Python and its use in data science and machine learning applications. The objectives are: - To describe the theoretical underpinnings of Python's exception handling. - To examine common root causes of errors in data science and machine learning processes. - To determine best practices to establish robust exception handling methods. - To illustrate how efficient exception handling enhances ML systems' dependability, reliability, and maintainability.

### 1.3 Scope and Structure of the Assignment

The current research excludes low-level systems programming and non-Python ML frameworks in order to emphasize Python-based data science and machine learning applications. The assignment is structured as follows: Section 2 introduces the fundamentals of exception handling in Python. Section 3 explores typical error sources in data science pipelines. Section 4 discusses best practices and design patterns. Section 5 examines exception handling in ML-specific contexts. Section 6 addresses logging, monitoring, and production considerations. Section 7 discusses ethical and reproducibility implications. Section 8 concludes the paper.

## 2. Fundamentals of Exception Handling in Python

### 2.1 Concept of Exceptions in Programming

Runtime incidents also called as exceptions interfere having the regular flow of program execution. They show that something out of the ordinary has occurred and requires careful handling(Lutz, 2013). In high-level languages like Python, exceptions provide a structured way to separate error-handling logic from crucial functionality, improving code readability and maintainability(McConnell, 2004).

In contrast to error codes, exceptions develop automatically until they are handled or result in program termination, which needs to be explicitly confirmed. This propagation strategy is particularly useful in complex data science workflows when functions are deeply nested and manual error checking is not feasible(Gamma et al., 1995).

### 2.2 Python’s Exception Hierarchy

Python uses objects inherited from the built-in BaseException class to implement exceptions.  The standard exception hierarchy includes commonly used exceptions such as SyntaxError, RuntimeError, IndexError, and IOError (Python Software Foundation, 2024). Developers can identify specific errors while letting others proliferate because to this structure.

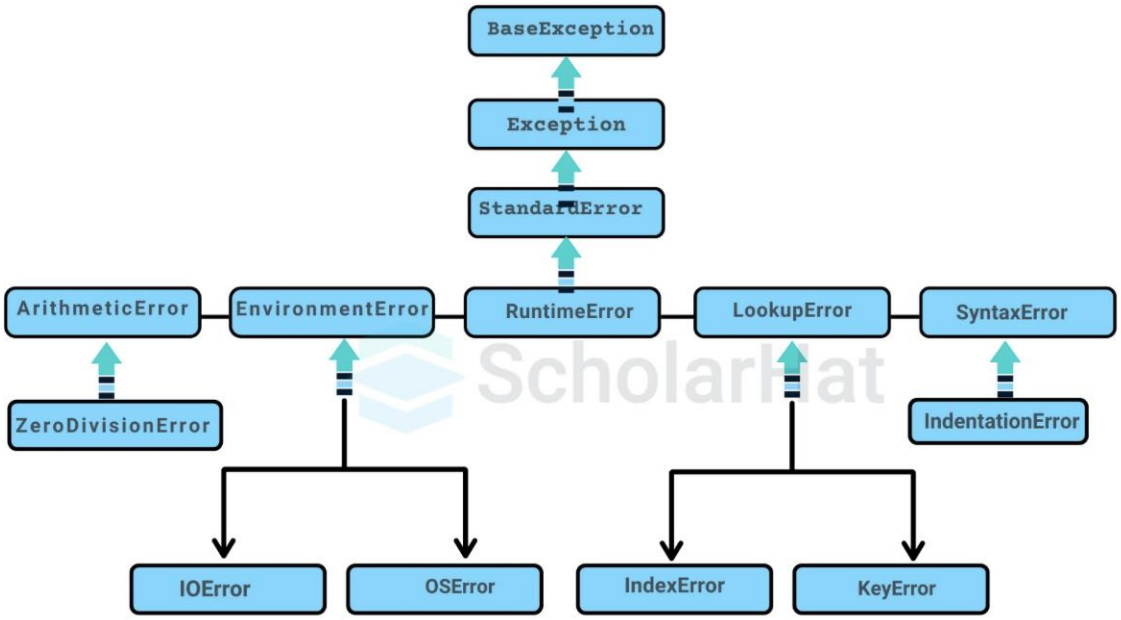



Figure 1: Python Exceptions Hierarchy - Image Source

For data science applications, where various error categories can be used for various remediation techniques, the organized hierarchy is crucial. For instance, a ValueError during preprocessing might indicate corrupted input data, whereas a FileNotFoundError during the loading of data might call for user interaction.

## 2.3 Syntax and Core Constructs

For handling exceptions, Python offers the try, except, else, and finally constructs. While except blocks specify handlers for exception types, try blocks include code that may raise an exception. Finally, regardless of the result, the optional else block runs if there is no exception.

In data-driven code, where presumptions about data quality frequently fail, this approach emphasizes explicit handling of expected failure scenarios (Jake VanderPlas, 2016).

### 2.4 Custom Exceptions

By subclassing Exception, Python programmers can create unique exception classes. By expressing domain-specific failure modes like InvalidDatasetError or ModelConvergenceError, custom exceptions enhance semantic clarity.

In machine learning systems, Custom exceptions facilitate more relevant error reporting and easier debugging, particularly when non-developer stakeholders like data analysts or decision-makers utilize the systems (Amershi et al., 2019).

## 3. Errors Sources in Data Science and Machine Learning Workflows

### 3.1 Data Collection and Ingestion Errors

Data ingestion from diverse sources, such databases, APIs, or flat files, is frequently the first step in data science pipelines. Schema mismatches, corrupted records, missing files, and network failures are examples of common problems (Knaflic, 2020). If exceptions are ignored at this point, incomplete datasets may be silently processed, jeopardizing subsequent analysis.

After data collection, if there is any null values it can create ambiguity in the output so it has to be handled correctly through exceptions.

```python
import pandas as pd

# Load dataset
df = pd.read_csv('data.csv')
df.head()
```

| | Type_of_Renewable_Energy | Installed_Capacity_MW | Energy_Production_MWh | Energy_Consumption_MWh | Energy_Storage_Capacity_MWh |
|---|---|---|---|---|---|
| **0** | 4 | 93.423205 | 103853.2206 | 248708.4892 | 2953.248771 |
| **1** | 4 | 590.468942 | 190223.0649 | 166104.1642 | 5305.174042 |
| **2** | 1 | 625.951142 | 266023.4824 | 424114.6308 | 2620.192622 |
| **3** | 1 | 779.998728 | 487039.5296 | 308337.7316 | 1925.250307 |
| **4** | 3 | 242.106837 | 482815.0856 | 360437.7705 | 3948.945383 |

Source: Own representation

By ensuring that data ingestion problems are identified early and conveyed explicitly, effective exception handling stops erroneous assumptions from spreading throughout the pipeline. (Sculley et al., 2015).

### 3.2 Data Preprocessing and Feature Engineering Errors

Data quality has a significant impact on preprocessing operations including encoding, imputation, and normalization. Common errors include division by zero, invalid type conversions, and unexpected null values (Geron, 2019).

Inadequate exception handling could cause the pipeline to crash or, worse, result in unexpectedly wrong outputs. Developers can use fallback techniques, like default values or data validation checks, by explicitly handling exceptions.

### 3.3 Model Training and Evaluation Errors

Other sources of mistake introduced by machine learning algorithms include non-convergence, numerical instability, and incompatible hyperparameters. Certain exceptions are raised by libraries like scikit-learn and TensorFlow to indicate these problems (Pedregosa et al., 2011; Abadi et al., 2016).

Diagnosing model behavior and preventing deceptive evaluation findings depend on appropriately identifying and understanding these deviations.

### 3.4 Deployment and Production Errors

In production environments, ML models are required to deal with operational limitations and unpredictable inputs. Data drift, resource depletion, and integration issues can all lead to errors. In deployed systems, graceful degradation is supported by robust exception handling, enabling systems to fail safely rather than catastrophically. (Humble & Farley, 2010).

### 3.5 Illustrative Python Code Examples for Common Error Scenarios

This article provides specific Python code examples that illustrate efficient exception handling in common data science and machine learning settings to bring the theoretical

concept to life. In order to emphasize fundamental ideas rather than library-specific optimizations, the examples have been purposefully simplified.

**Example 1: Safe Data Loading with Explicit Exception Handling**

```
import pandas as pd

try:
    df = pd.read_csv("data/training_data.csv")
except FileNotFoundError as e:
    raise FileNotFoundError("The specified data file could not be found.")
from e
except pd.errors.ParserError as e:
    raise ValueError("The data file is malformed and cannot be parsed.") fr
om e
else:
    print("Data loaded successfully.")
```

This illustration shows how some exceptions enhance mistake transparency during data import, a crucial first stage in any machine learning pipeline (McKinney, 2017).

**Example 2: Handling Invalid Data During Preprocessing**

```
import numpy as np

def normalize_column(column):
    try:
        return (column - np.mean(column)) / np.std(column)
    except ZeroDivisionError:
        raise ValueError("Standard deviation is zero; normalization is not
possible."
```

Here, exception handling prevents silent numerical errors that could invalidate downstream model training (Harris et al., 2020).

**Example 3: Custom Exceptions for Domain-Specific Errors**

```python
class ModelTrainingError(Exception):
    """Raised when a machine learning model fails to train correctly."""
    pass

from sklearn.linear_model import LogisticRegression

try:
    model = LogisticRegression(max_iter=1)
    model.fit(X_train, y_train)
except Exception as e:
    raise ModelTrainingError("Model failed to converge during training.")
from e
```

Semantic meaning is encoded by custom exceptions, which improves debugging and stakeholder communication (Lutz, 2013).

## 4. Best Practices for Effective Exception Handling in Python

### 4.1 Catching Specific Exceptions

Catching exceptions that are too general (such as except Exception) makes debugging more difficult and hides the underlying cause of issues. Finding the most precise exception is advised by best practices.

This technique increases transparency in data science code and guarantees that unforeseen mistakes are not disregarded (Wilson et al., 2014).

### 4.2 Avoiding Silent Failures

It is generally considered an anti-pattern to silently suppress exceptions, such as by employing empty unless blocks (McConnell, 2004). Silent failures can compromise scientific rigor and invalidate experimental results in machine learning scenarios.

Accountability and traceability are ensured by explicitly logging or raising exceptions (Hunt & Thomas, 1999).

### 4.3 Using Exceptions for Exceptional Conditions

For regular control flow, exceptions shouldn't be used. Code performance and clarity are can be decreased when predicted situations are handled using exceptions (Gamma et al., 1995).

While exceptions should indicate genuinely anomalous states, conditional checks in data pipelines should accommodate expected data shifts.

### 4.4 Integrating Validation and Exception Handling

By avoiding invalid states from occurring, input validation enhances exception handling. Python-based systems frequently use libraries like Cerberus and Pedantic for data validation (Pedregosa et al., 2011). Code that is more reliable and self-documenting is produced when structured exception handling and validation are combined.

## 5. Beneficial Exception Handling in Machine Learning Libraries

### 5.1 NumPy and SciPy

According to Harris et al. (2020), NumPy operations may throw exceptions pertaining to shape mismatches, improper operations, or numerical overrun. In scientific computing, where numerical errors can subtly skew results, handling these exceptions is considered crucial.

### 5.2 pandas

For improper type operations, missing columns, and indexing issues, the pandas library produces exceptions. Reliable data transformation and manipulation are supported by appropriate management of these exceptions (McKinney, 2017).

### 5.3 scikit-learn

Exceptions are implemented by Scikit-learn to indicate erroneous parameter combinations and incorrect model fitting. Automated experimentation frameworks may

circumvent unsuccessful runs without terminating entire studies simply catching these exceptions (Pedregosa et al., 2011).

### 5.4 Deep Learning Frameworks

Runtime exceptions about hardware acceleration, memory use, and computational graphs are raised by frameworks like TensorFlow and PyTorch (Abadi et al., 2016; Paszke et al., 2019).

Managing distributed systems and long-running training jobs requires effective exception management.

## 6. Logging, Monitoring, and Production Considerations

### 6.1 Advanced Exception Handling with Logging (along with Production-Oriented Example)

Debugging and auditing require contextual information, which is provided by logging exceptions. Configurable logging levels and structured output are supported by Python's built-in logging module. Logs are the main source of operational information in production machine learning systems (Python Software Foundation, 2024).

Exceptions should almost never be handled in production-grade data science and machine learning systems without being documented. Logging provides durable, time-stamped logs that are essential for debugging, auditing, and system health monitoring (Humble & Farley, 2010).

**Example 4: Exception Handling Integrated with Logging**

```python
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    filename="ml_pipeline.log"
)

try:
    result = 10 / 0
except ZeroDivisionError as e:
    logging.error("Division by zero occurred during feature computation",
exc_info=True)
    raise
```

**Expected Output (Log File):**

```
2026-01-10 14:22:31 - ERROR - Division by zero occurred during feature
computation
Traceback (most recent call last):
  File "pipeline.py", line 10, in <module>
    result = 10 / 0
ZeroDivisionError: division by zero
```

This example shows how exceptions can be preserved using full stack traces, which makes post-mortem analysis and continuous ML system improvement easier (Python Software Foundation, 2024).

### 6.2 Exception Handling in Deep Learning Training Loops

Deep learning models are frequently trained in lengthy procedures where hardware constraints, faulty tensors, or numerical instability may cause failures. Training jobs can end gracefully while maintaining diagnostic data when exception handling is done correctly (Paszke et al., 2019).

To guarantee prompt failure reaction, monitoring and alerting systems should be connected with exception handling. For ML systems that function independently, this is especially crucial (Amershi et al., 2019).

**Example 5: Safe Training Loop with Exception Handling (PyTorch)**

```
import torch

try:
    model = torch.nn.Linear(10, 1)
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

    for epoch in range(5):
        optimizer.zero_grad()
        outputs = model(torch.randn(3, 5))  # Intentional shape mismatch
        loss = outputs.sum()
        loss.backward()
        optimizer.step()
except RuntimeError as e:
    print("Runtime error during model training:", e)
```

**Expected Console Output:**

```
Runtime error during model training: mat1 and mat2 shapes cannot be multiplied
```

According to Abadi et al. (2016), this method avoids quiet crashes and offers prompt feedback on architectural or data-shape discrepancies.

### 6.3 Supporting Reproducibility Through Controlled Failure Handling

Reproducibility is a central concern in scientific computing and machine learning research. Exception handling contributes to reproducibility by ensuring that failed experiments are explicitly recorded rather than silently ignored (Wilson et al., 2014).

The ability of a system to continue functioning at a reduced level in the face of faults is known as graceful degradation. Such behavior is made possible in large part via exception handling (Hunt & Thomas, 1999).

**Example 6: Exception Handling in Experimental Pipelines**

```python
import random
import numpy as np

def run_experiment(seed):
    try:
        random.seed(seed)
        np.random.seed(seed)
        value = np.log(-1)  # Invalid mathematical operation
        return value
    except FloatingPointError as e:
        return f"Experiment failed due to numerical error: {e}"

print(run_experiment(42))
```

**Expected Output:**

```
Experiment failed due to numerical error: invalid value encountered in log
```

Researchers can prevent biased result reporting and preserve transparent experimental records by clearly documenting and reporting failures (Sculley et al., 2015).

## 7. Ethical, Reproducibility, and Scientific Implications

Reproducibility may be compromised by insufficient exception handling, which permits tests to finish in spite of concealed mistakes. This goes against ethical research norms and undermines the validity of scientific findings (Wilson et al., 2014).

Unhandled exceptions in deployed machine learning systems might have negative ethical effects, particularly across sensitive domains. Therefore, transparent and reliable error handling is not just a technical resolution but also an ethical duty (Amershi et al., 2019).

## 8. Conclusion

This assignment has shown that reliable Python-based data science and machine learning systems depend on effective exception handling. It is clear from examining theoretical concepts, typical error causes, and best practices that structured exception handling improves dependability, maintainability, repeatability, and ethical accountability.

Disciplined exception management must be regarded as a crucial component of both academic methodology and professional practice as machine learning systems continue to evolve in complexity and societal impact. Further research could look at standardized mistake taxonomies for machine learning systems and automated tools for exception analysis.

## 9 References

Abadi, M., Barham, P., Chen, J., et al. (2016). TensorFlow: A system for large-scale machine learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI). https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

Amershi, S., Begel, A., Bird, C., et al. (2019). Software engineering for machine learning: A case study. IEEE Transactions on Software Engineering, 45(12), 1151–1168. https://doi.org/10.1109/TSE.2019.2913101

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Humble, J., & Farley, D. (2010). Continuous delivery. Addison-Wesley.

Hunt, A., & Thomas, D. (1999). The pragmatic programmer. Addison-Wesley.

VanderPlas, J. (2016). Python data science handbook. O'Reilly Media.

Kelleher, J. D., Mac Namee, B., & D'Arcy, A. (2020). Fundamentals of machine learning for predictive data analytics. MIT Press.

Knaflic, S. N. (2020). Storytelling with data. Wiley.

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

McConnell, S. (2004). Code complete (2nd ed.). Microsoft Press.

McKinney, W. (2017). Python for data analysis (2nd ed.). O'Reilly Media.

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Python Software Foundation. (2024). Python documentation. https://docs.python.org/3/

Sculley, D., Holt, G., Golovin, D., et al. (2015). Hidden technical debt in machine learning systems. Advances in Neural Information Processing Systems, 28. https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Van Rossum, G., & Drake, F. L. (2009). Python 3 reference manual. Addison-Wesley.

Wilson, G., Aruliah, D. A., Brown, C. T., et al. (2014). Best practices for scientific computing. PLoS Biology, 12(1), e1001745. https://doi.org/10.1371/journal.pbio

# 10 APPENDIX

## 10.1 About Code

**Github Link: https://github.com/ZaibaKhan2025/IUBH_programming_with_python**

Note: The practical work only has one file with titled task.py

Task.py has two portions of code. The first patch of code helps to create database and tables.

https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter16.04-Least Squares-Regression-in-Python.html for least square method.
https://www.w3schools.com/python/ used basics as a reference.

https://docs.bokeh.org/en/latest/ for visualization.

## 10.2 Code:

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
import mysql.connector
import sys
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit
import mysql.connector
from mysql.connector import Error
from bokeh.plotting import figure, show, output_file
from bokeh.models import ColumnDataSource, HoverTool
mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    password=""
)

# Queries for database:

mycursor = mydb.cursor()

sql_queries = [
    "DROP DATABASE IF EXISTS python_assignment_zaiba_khan",
    "CREATE DATABASE python_assignment_zaiba_khan",
    "CREATE TABLE train (id INT AUTO_INCREMENT PRIMARY KEY, x float, y1 float, y2 float, y3 float, y4 float)",
    "CREATE TABLE test (id INT AUTO_INCREMENT PRIMARY KEY, x float, y float)",
    "CREATE TABLE best_fit_func (id INT AUTO_INCREMENT PRIMARY KEY, x float, y float, choosen_func float)",
    "CREATE TABLE ideal (id INT AUTO_INCREMENT PRIMARY KEY, x float)",
    "CREATE TABLE mapping (id INT AUTO_INCREMENT PRIMARY KEY, x float, y float, ideal_x float, ideal_y float, deviation float)",
    *[f"ALTER TABLE ideal ADD COLUMN y{i+1} float;" for i in range(0, 50)]
]

mycursor.execute(sql_queries[0])
mycursor.execute(sql_queries[1])
```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python

mycursor.execute(sql_queries[0])
mycursor.execute(sql_queries[1])

mydb_1 = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="python_assignment_zaiba_khan"
)

mycursor_1 = mydb_1.cursor()

for i, table in enumerate(sql_queries):
    if i > 1:
        mycursor_1.execute(table)

print("Database and Tables Created Successfully !")
# Queries for database end:

db_connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="python_assignment_zaiba_khan"
)

mycursor_1 = db_connection.cursor()
mycursor_1.execute("show databases")
databases = mycursor_1.fetchall()
database_name = [i[0] for i in databases]

```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python

if 'python_assignment_zaiba_khan' in database_name:
    print(' ')
else:
    print('Database not exist, please read the instructions given !')
    sys.exit()

# CSV files to veriables
def load_data(file_path):
    data = pd.read_csv(file_path)
    x_val = data['x'].values
    y_cols = [col for col in data.columns if col.startswith('y')]
    y_val = data[y_cols].values
    return x_val, y_val

# ideal function
def ideal_function(x, a, b):
    return a * x + b

# Fit ideal function to training data
def fit_ideal_functions(training_data, ideal_functions):
    best_function_val = []
    min_deviation = float('inf')
    min_sum_squared_deviations_arry = []
    x_train, y_train = training_data

    j = 0
    for y in y_train:
        if j < 4:
            for y_ideal in ideal_functions:
                # Fit only the first column of y_train
                params, _ = curve_fit(ideal_function, x_train, y_train[:, j])
```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
def fit_ideal_functions(training_data, ideal_functions):
                    params, _ = curve_fit(ideal_function, x_train, y_train[:, j])
                    y_pred = ideal_function(x_train, *params)
                    # Calculate deviation only for the first column
                    sum_squared_deviations = np.sum((y_train[:, j] - y_pred) ** 2)

                    if sum_squared_deviations < min_deviation:
                        min_deviation = sum_squared_deviations
                        best_function_params = params
                j = j+1
                best_function_val.append(best_function_params)
                min_sum_squared_deviations_arry.append(min_deviation)
    return best_function_val, min_sum_squared_deviations_arry
    # put these values into the best_fit_function table

# Map test data to chosen ideal functions
def map_test_data(test_data, chosen_functions):
    x_test, y_test = test_data
    mappings = []

    for x, y in zip(x_test, y_test):
        deviations = []
        for params in chosen_functions:
            y_pred = ideal_function(x, *params)
            deviation = np.abs(y - y_pred)
            deviations.append(deviation)
        min_deviation = min(deviations)
        if min_deviation < np.sqrt(2) * np.max(deviations):
            best_fit_index = deviations.index(min_deviation)
            best_fit_params = chosen_functions[best_fit_index]
            mappings.append((x, y, best_fit_params, min_deviation))
        else:
```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
def map_test_data(test_data, chosen_functions):
            mappings.append((x, y, best_fit_params, min_deviation))
        else:
            mappings.append((x, y, None, None))
    # print(mappings[0])
    # sys.exit()
    return mappings


x_train, y_train = load_data("train.csv")  # training data (A)
ideal_functions = load_data("ideal.csv")  # ideal data (C)
x_test, y_test = load_data("test.csv")  # test data

try:
    train_data_list = [
        (x, y1, y2, y3, y4)
        for x, y1, y2, y3, y4 in zip(x_train, y_train[:, 0], y_train[:, 1], y_train[:, 2], y_train[:, 3])
    ]
    mycursor_1.executemany(
        "INSERT INTO train (x, y1, y2, y3, y4) VALUES (%s, %s, %s, %s, %s)",
        train_data_list
    )
    db_connection.commit()

    test_data_list = [
        (x, y)
        for x, y in zip(x_test, y_test[:, 0])
    ]
    mycursor_1.executemany(
        "INSERT INTO test (x, y) VALUES (%s, %s)",
        test_data_list
    )
```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
    )
    db_connection.commit()

    x_ideal, y_ideal = ideal_functions
    ideal_data_list = [
        (x, *y_row)
        for x, y_row in zip(x_ideal, y_ideal)
    ]
    mycursor_1.executemany(
        "INSERT INTO ideal (x, " + ", ".join(
            [f"y{i+1}" for i in range(50)]) + ") VALUES (" + ", ".join(["%s"] * 51) + ")",
        ideal_data_list
    )
    db_connection.commit()

except Error as e:
    # raise Exception("Sorry")
    print(f"Something Happened {e}")
finally:
    print("xxxxxxxxxxxxxxxxx")

# Choose the four best fitting ideal functions
chosen_functions = fit_ideal_functions((x_train, y_train), ideal_functions)

# Map test data to chosen ideal functions
mappings = map_test_data((x_test, y_test), chosen_functions[0])

print("Extrected ideal function Data :")
print("S.No_______x_________________y_______________chosen_val")
for i, params in enumerate(chosen_functions[0]):
    print(f" {i+1}: {params[0]}, {params[1]}, {chosen_functions[1][i]}")
    data_to_insert = [(params[0], params[1], chosen_functions[1][i])]
```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
    data_to_insert = [(params[0], params[1], chosen_functions[1][i])]
    mycursor_1.executemany(
        "INSERT INTO best_fit_func (x,y,choosen_func) VALUES (%s, %s, %s)", data_to_insert)
db_connection.commit()

print()
print("Mappings Function :")
print()
print("_x____|___y____|____fit func [A]__|___fit func [B]___|___Deviation")
for i, (x, y, best_fit_params, deviation) in enumerate(mappings):
    if best_fit_params is not None:
        print(x, y[0], best_fit_params[0], best_fit_params[1], deviation[0])
        data_to_insert = [(x, y[0], best_fit_params[0],
                           best_fit_params[1], deviation[0])]
        mycursor_1.executemany(
            "INSERT INTO mapping (x,y,ideal_x,ideal_y,deviation) VALUES (%s, %s, %s, %s, %s)", data_to_insert)
    else:
        print(f"Data point {
            i+1}: x={x}, y={y}, No best fit found within the deviation threshold")
db_connection.commit()

# Graphing
def plot_data_with_bokeh(x_train, y_train, x_test, y_test, chosen_functions, mappings):
    try:
        p = figure(title="Training Data, Ideal Functions, Test Data, and Mappings",
                   x_axis_label='x', y_axis_label='y', width=800, height=600)

        # Plot training data
        for i in range(y_train.shape[1]):
            p.scatter(x_train, y_train[:, i], legend_label=f'Training y{
                i+1}', color='green', size=6, alpha=0.6)

```

task.py

D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
def plot_data_with_bokeh(x_train, y_train, x_test, y_test, chosen_functions, mappings):

        # Plot ideal functions
        x_range = np.linspace(min(x_train), max(x_train), 500)
        for i, params in enumerate(chosen_functions[0]):
            y_range = ideal_function(x_range, *params)
            p.line(x_range, y_range, legend_label=f'Ideal Function {
                    i+1}', color='pink', line_width=2)

        # Plot test data
        for i in range(y_test.shape[1]):
            p.scatter(x_test, y_test[:, i], legend_label=f'Test Data y{
                    i+1}', color='black', size=6, alpha=0.6)

        # Plot mappings with deviation
        mapped_x = [x for x, y, function_idx,
                    deviation in mappings if function_idx is not None]

        mapped_y = [y for x, y, function_idx,
                    deviation in mappings if function_idx is not None]
        deviations = [deviation for x, y, function_idx,
                    deviation in mappings if function_idx is not None]
        source = ColumnDataSource(
            data=dict(x=mapped_x, y=mapped_y, deviation=deviations))

        p.scatter('x', 'y', source=source, color='red', size=10,
                  alpha=0.6, legend_label='Mapped Test Data')

        hover = HoverTool()
        hover.tooltips = [("x", "@x"), ("y", "@y"),
                          ("deviation", "@deviation")]
        p.add_tools(hover)
```

task.py

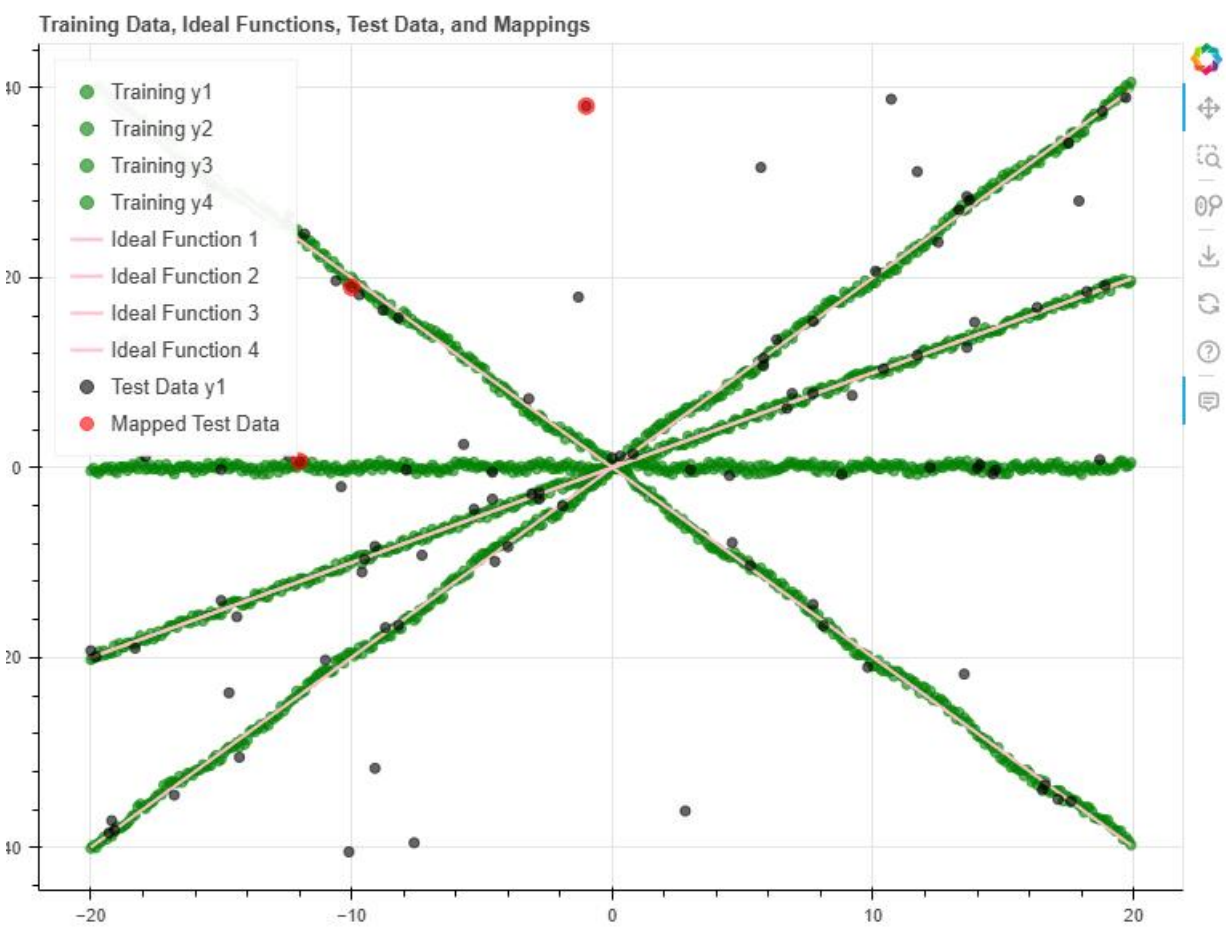
D: > IU University of applied science > 1st semester > quarter 1 > programming with python > zaiba > task.py > ...

```python
def plot_data_with_bokeh(x_train, y_train, x_test, y_test, chosen_functions, mappings):

        p.legend.location = "top_left"
        output_file("mapping.html")
        show(p)
    except Error as e:
        print(f"some kind of error {e}")

plot_data_with_bokeh(x_train, y_train, x_test, y_test,
                     chosen_functions, mappings)

def plot_data_xtrain_ytrain(x,y):
    p = figure(title="Training Dataset", x_axis_label="x", y_axis_label="y")

    p.line(x, y[:, 0], legend_label="x y1", color="yellow", line_width=2)
    p.line(x, y[:, 1], legend_label="x y2", color="green", line_width=2)
    p.line(x, y[:, 2], legend_label="x y3", color="blue", line_width=2)
    p.line(x, y[:, 3], legend_label="x y4", color="red", line_width=2)

    output_file("train.html")
    show(p)
plot_data_xtrain_ytrain(x_train,y_train)

def plot_data_xtest_ytest(x,y):

    p = figure(title="Test Dataset", x_axis_label="x", y_axis_label="y")

    p.line(x, y[:,0], legend_label="x y", color="yellow", line_width=2)

    output_file("test.html")
    show(p)
plot_data_xtest_ytest(x_test, y_test)
```

## 10.3 Result:

```
C:\Windows\System32\cmd.exe

D:\IU University of applied science\1st semester\quarter 1\programming with python\assignment\1st solution>queries.py
Database and Tables Created Successfully !

D:\IU University of applied science\1st semester\quarter 1\programming with python\assignment\1st solution>task.py
true
Data insertion completed successfully.
Enjoy !
Chosen ideal function parameters:
S.No        x                    y                  chosen_val
 1: -1.9978981440449517, 0.007771132716573925, 87.11627242584481
 2: 1.998638834500751, 0.00569289893183722, 86.13857933247337
 3: 0.9996282494243194, -0.0016468939684690653, 29.853399197190416
 4: 0.9996282494243194, -0.0016468939684690653, 29.853399197190416

Mappings:

 x       y        Best fit (A)     Best fit  (B)      Deviation
17.5 34.16104 1.998638834500751 0.00569289893183722 0.8208325026949836
0.3 1.2151024 1.998638834500751 0.00569289893183722 0.6098178507179374
-8.7 -16.843908 1.998638834500751 0.00569289893183722 0.5385569612246961
-19.2 -37.17087 1.998638834500751 0.00569289893183722 1.1973027234825864
-11.0 -20.263054 1.998638834500751 0.00569289893183722 1.7162802805764237
0.8 1.4264555 1.998638834500751 0.00569289893183722 0.178148466532438
14.0 -0.06650608 0.9996282494243194 -0.0016468939684690653 14.059654677972002
-10.4 -2.007094 0.9996282494243194 -0.0016468939684690653 8.390686687981392
-15.0 -0.20536347 0.9996282494243194 -0.0016468939684690653 14.790707165333261
5.8 10.711373 1.998638834500751 0.00569289893183722 0.8864251390361932
-7.6 -39.4954 1.998638834500751 0.00569289893183722 24.31143775672613
-19.8 -19.915014 0.9996282494243194 -0.0016468939684690653 0.12072776743000446
```

## 10.4 Visualization:



Thank you ☺